AI ASSISTED CODING

Name: P.Swaran Raj **Roll.No:** 2303A52247

Batch: 37 **Assignment -** 15.2

Task Descriptor-1: (Setup Fast API Backend)

Installed packages

Pip install fastapi uvicorn

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                pip
PS C:\Users\pulla\OneDrive\Documents\SRU\AI Asisted Coding> pip install fastapi
  Downloading typing_inspection-0.4.2-py3-none-any.whl.metadata (2.6 kB)
Collecting annotated-doc>=0.0.2 (from fastapi)
  Downloading annotated_doc-0.0.4-py3-none-any.whl.metadata (6.6 kB)
Collecting annotated-types>=0.6.0 (from pydantic>=2.7.0->fastapi)
  Using cached annotated_types-0.7.0-py3-none-any.whl.metadata (15 kB)

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
ERROR: Operation cancelled by user
PS C:\Users\pulla\OneDrive\Documents\SRU\AI Asisted Coding> pip install uvicorn
Collecting uvicorn
```

```
PS C:\Users\pulla\OneDrive\Documents\SRU\AI Asisted Coding> pip install fastapi uvicorn
PS C:\Users\pulla\OneDrive\Documents\SRU\AI Asisted Coding> pip install fastapi uvicorn
Collecting fastapi
  Using cached fastapi-0.135.1-py3-none-any.whl.metadata (30 kB)
Requirement already satisfied: uvicorn in c:\users\pulla\appdata\local\programs\python\python313
\lib\site-packages (0.41.0)
Collecting starlette>=0.46.0 (from fastapi)
  Using cached starlette-0.52.1-py3-none-any.whl.metadata (6.3 kB)
Collecting pydantic>=2.7.0 (from fastapi)
  Using cached pydantic-2.12.5-py3-none-any.whl.metadata (90 kB)
Requirement already satisfied: typing-extensions>=4.8.0 in c:\users\pulla\appdata\local\programs
\python\python313\lib\site-packages (from fastapi) (4.13.2)
Collecting typing-inspection>=0.4.2 (from fastapi)
```

main.py > ...

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}
```

Output:

```
Pretty-print

{"message":"Welcome to AI-powered FastAPI"}
```

Observation:

The FastAPI framework allows developers to quickly build backend APIs with minimal configuration.AI-assisted code generation helped create the basic backend structure efficiently.Using Uvicorn, the API server was successfully executed and the endpoint returned the expected JSON response.

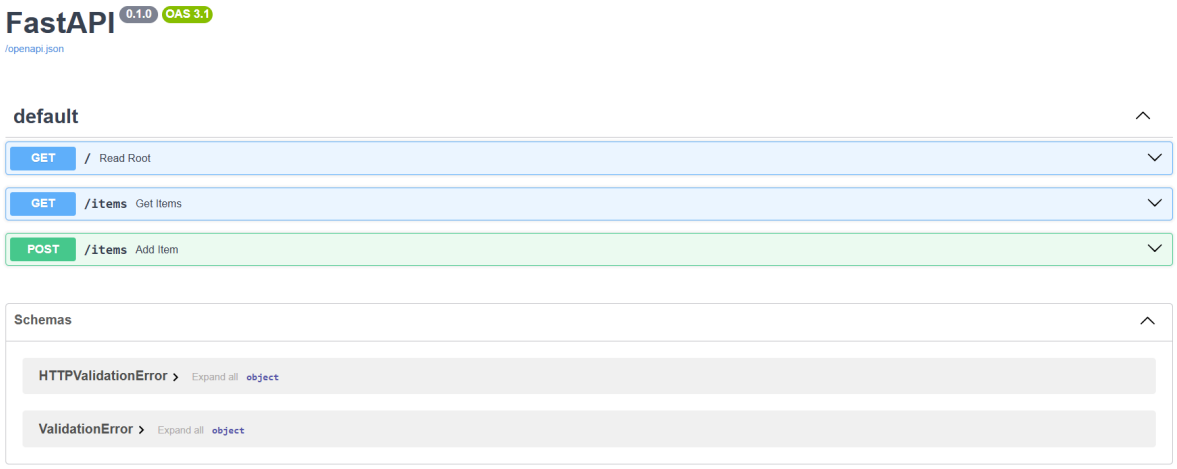
Task Descriptor- 2: (API – Create and Read Items)

main.py > read_root

```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}

items = []
@app.get("/items")
def get_items():
    return items
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
```

Output:



Explanation:

In this task, two FastAPI endpoints were created to manage items. The GET `/items` endpoint retrieves all stored items, while the POST `/items` endpoint allows adding a new item to a Python list used as temporary storage.

Observation:

The API successfully handled item creation and retrieval.
Swagger UI helped test the endpoints and view JSON responses easily.

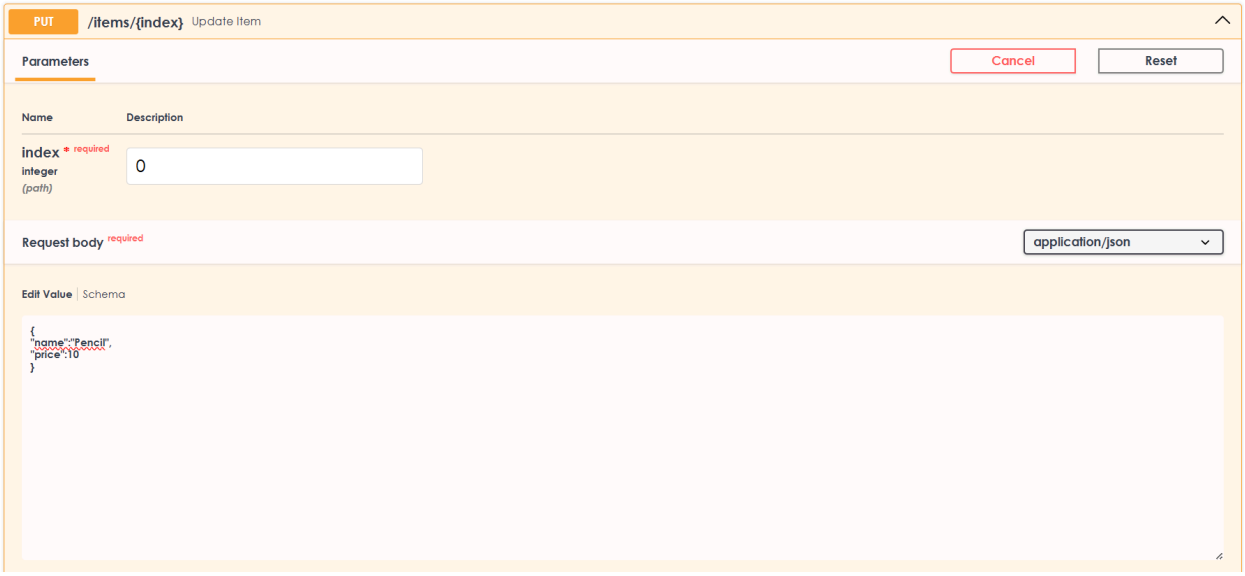
Task Descriptor- 3: (Update Existing Item)

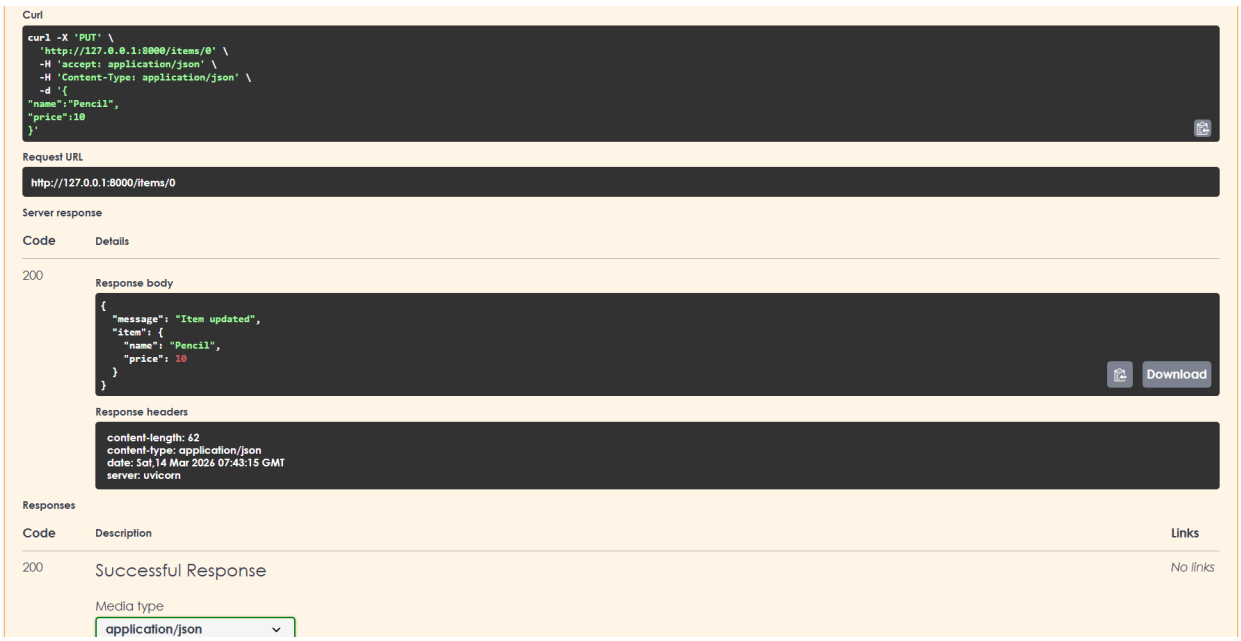
```python
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}

items = []
@app.get("/items")
def get_items():
    return items
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}

@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index >= len(items):
        return {"error": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}
```

Output:





Explanation:

The PUT endpoint `/items/{index}` is used to update an existing item in the list.It takes the item index from the URL and replaces the old item with the new data.If the index does not exist, the API returns an "Item not found" error.

Output:

Request: `PUT /items/0`

{

"name": "Pencil",

"price": 10
}
Response:
{
"message": "Item updated",
"item": {"name":"Pencil","price":10}
}

Observation:

The API successfully updated the item stored at the specified index.This demonstrates the Update operation of CRUD using the PUT method in FastAPI.

Task Descriptor- 4: (Delete Item)

```python
    return {"message": "Welcome to AI-powered FastAPI"}

items = []
@app.get("/items")
def get_items():
    return items
@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}

@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index >= len(items):
        return {"error": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}

@app.delete("/items/{index}")
def delete_item(index: int):
    if index >= len(items):
        return {"error": "Item not found"}
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```

Output:

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/items/0' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
"message":"Item removed",
"item":{"name":"Pencil","price":10}
}'
```

Request URL

http://127.0.0.1:8000/items/0

Server response

Code	Details
200	Response body

```
{
  "error": "Item not found"
}
```

Download

Response headers

```
content-length: 26
content-type: application/json
date: Sat,14 Mar 2026 07:57:06 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

DELETE /items/{index} Delete Item

Parameters

Cancel

Name	Description
index * required integer (path)	0

Execute Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/items/0' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/items/0

Server response

Code	Details
200	Response body

```
{
  "message": "Item removed",
  "item": {
    "name": "Pencil",
    "price": 10
  }
}
```

Download

Response headers

```
content-length: 62
content-type: application/json
date: Sat,14 Mar 2026 07:56:56 GMT
server: uvicorn
```

Explanation:

The DELETE endpoint `/items/{index}` is used to remove an item from the list using its index.

It checks whether the given index exists in the list.
If valid, the item is deleted and returned in the response.

Output:

Request: `DELETE /items/0`
Response:
{
"message":"Item removed",
"item":{"name":"Pencil","price":10}
}

Observation:

The API successfully deleted the item from the list.
This demonstrates the Delete operation of CRUD in FastAPI.

Task Descriptor -5: (Add Auto-Generated Documentation)

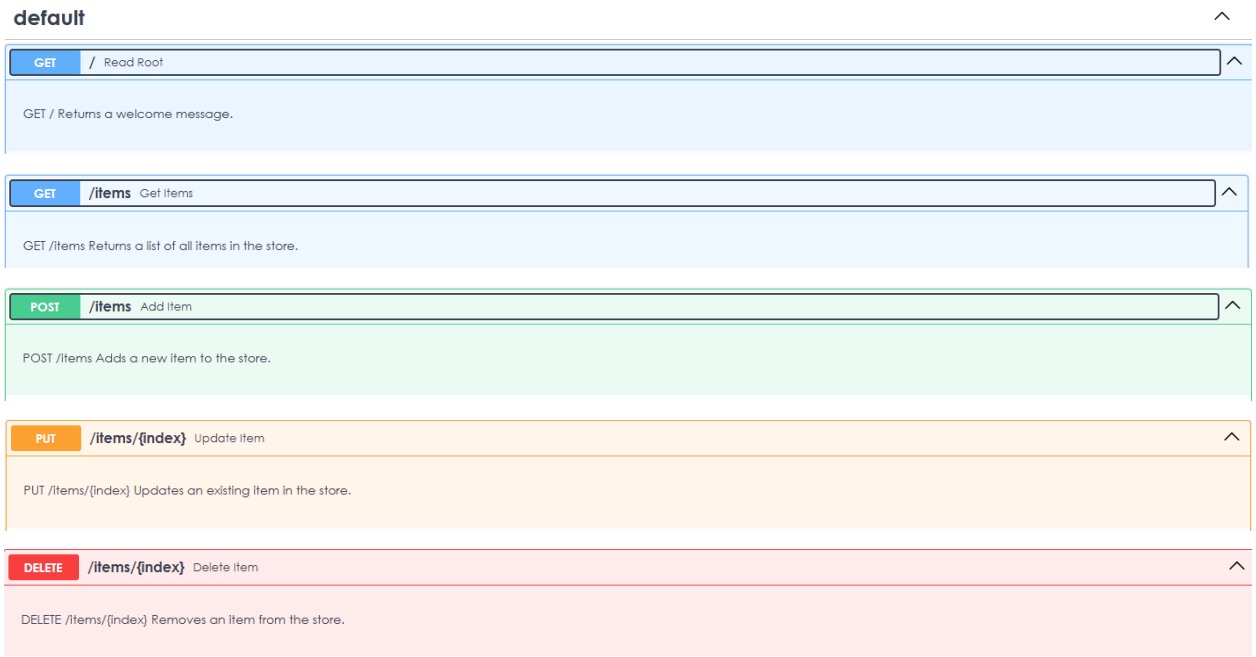
```python
def update_item(index: int, item: dict):
    if index >= len(items):
        return {"error": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}

@app.delete("/items/{index}")
def delete_item(index: int):
    if index >= len(items):
        return {"error": "Item not found"}
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}

@app.route('/items', methods=['GET'])
def get_items():
    """
    GET /items
    Returns a list of all items in the store.
    """
    return jsonify(items)
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Output:



Explanation:

In this task, inline comments and docstrings are added to the FastAPI endpoints.FastAPI automatically generates interactive API documentation using Swagger UI.This documentation can be accessed through the `/docs` endpoint.

Output:

Open in browser:

http://127.0.0.1:8000/docs

The Swagger page displays all endpoints like GET /items, POST /items, PUT /items/{index}, DELETE /items/{index} with request and response details.

Observation:

FastAPI automatically generated API documentation using Swagger UI. This allows users to easily view, understand, and test all API endpoints interactively.

Conclusion:

In this lab, we learned how to build a **RESTful API using FastAPI** and implement basic **CRUD operations** such as create, read, update, and delete. We also explored how **AI-assisted coding helps generate backend services quickly**. Additionally, FastAPI's **auto-generated Swagger documentation** made it easy to test and understand the API endpoints.